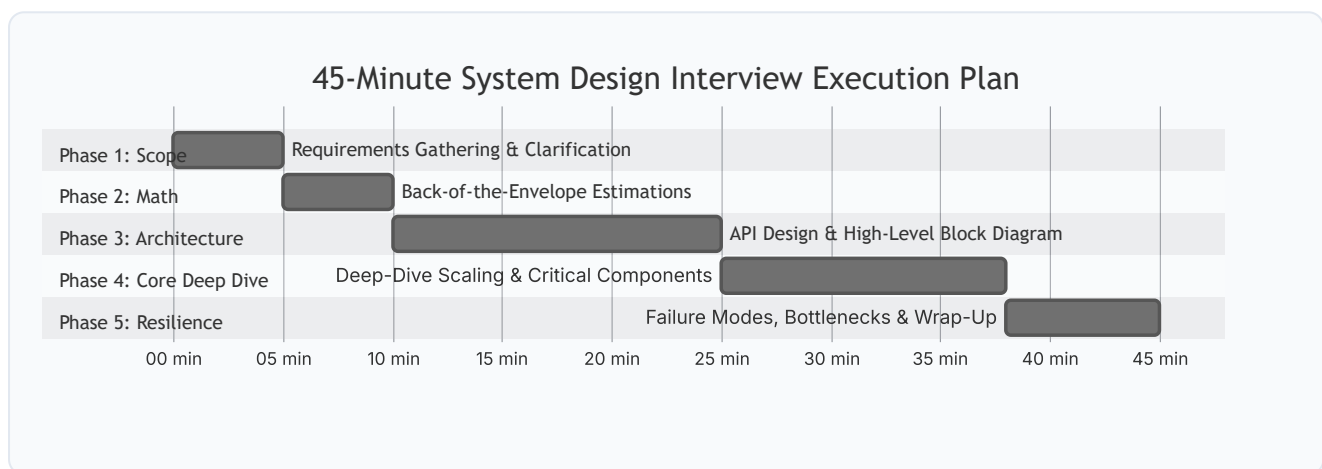


HLD Chapter 6: The 45-Minute System Design Interview Communication Playbook

Core Learning Objective: Master the exact pacing, structured dialogue, and technical storytelling required to earn Strong Hire ratings from FAANG/PBC interviewers.

1. The 45-Minute System Design Timeline



2. Phase-by-Phase Execution Guide

Phase 1: Requirements Gathering (Minutes 0 - 5)

- **Never assume requirements.** System design questions are intentionally underspecified (e.g., "Design Twitter").
- **Functional Requirements (FR):** Pinpoint the exact 3 or 4 features you will design.
 - *"Will users only post text, or also photos and videos? Can users follow each other? Do we need a live timeline feed?"*
- **Non-Functional Requirements (NFR):** Establish scale, availability, and latency SLAs.
 - *High Availability (99.99%) over Strong Consistency?*
 - *Low latency read queries (< 100ms at p99)?*

Phase 2: Back-of-the-Envelope Estimations (Minutes 5 - 10)

- Calculate Daily Active Users (DAU), Read QPS, Write QPS, Storage per day, and 5-Year Storage footprint.
- Calculate Cache memory requirements using the 80/20 rule.
- Conclude with the architectural character: *"This is a 10:1 read-heavy system requiring extensive CDN caching and replica sharding."*

Phase 3: High-Level Architecture Diagram (Minutes 10 - 25)

- Draw the end-to-end request lifecycle:
 1. Client (Mobile/Web) → DNS (Route53/Anycast) → CDN (Cloudflare/CloudFront) for static assets.
 2. Dynamic traffic → Load Balancer (ALB) → API Gateway (Authentication, Rate Limiting, SSL Termination).
 3. Microservices (Stateless application servers).
 4. Storage Layer: Cache (Redis cluster) → Primary DB (PostgreSQL/Cassandra) → Async Queues (Kafka).

Phase 4: The Deep Dive (Minutes 25 - 38)

- Interviewers evaluate your depth on **2 or 3 critical system bottlenecks**. Focus on:
 1. **Hot Partitions & Celebrity Problem:** (e.g. A user with 100M followers posts a tweet).
 2. **Concurrency & Race Conditions:** (e.g. Distributed locking on ticket reservations).
 3. **Data Partitioning Strategy:** (e.g. Consistent hashing by user ID or geolocation).

Phase 5: Bottlenecks & Failure Modes (Minutes 38 - 45)

- **Single Points of Failure (SPOF):** Show that every component has an active redundant replica.
- **Cross-Region Replication:** Explain multi-region active-active deployment or active-passive failover.
- **Observability:** Distributed tracing (OpenTelemetry), metrics (Prometheus), and alerting.

3. High-Scoring Behavioral Tactics (From Staff Interviewers)

1. **Keep Continuous Dialogue:** System design is a collaborative design session. If you go silent for 2 minutes while drawing, the interviewer feels detached. Vocalize your thought process: *"I am choosing Cassandra over PostgreSQL here because our write volume is 50,000 QPS and our query pattern is strictly key-value lookups by message ID..."*

2. **Never Rush into Components:** Junior candidates immediately draw Kafka, Redis, and Elasticsearch on the whiteboard within 2 minutes without justifying why. Senior candidates justify every box with mathematical estimation and trade-off analysis.
3. **Cross-Region Strategy is the Golden Touch:** Conclude by explaining how data replicates across US-East and EU-Central datacenters, handling data sovereignty (GDPR) and cross-ocean replication latency.

4. A Worked 45-Minute Timeline: "Design a Notification System"

The framework is easiest to learn from a concrete clock. Each row is a block with a deliverable; if you are behind, shorten the later blocks rather than skipping the earlier ones.

Minutes	Block	What you say or draw	Deliverable
0 to 5	Clarify	"Which channels? Delivery guarantee? Scale? Scheduled sends?"	A written list of functional and non-functional requirements
5 to 10	Estimate	250M notifications a day is about 3,000 per second average, with campaign bursts near 33,000 per second	Two or three numbers that shape the design
10 to 15	API and data	POST /notifications returns 202, idempotency key, preferences table	The contract and the main entities
15 to 25	High-level design	Producers, API, validation, priority queues per channel, workers, providers, dead-letter queue	One diagram, labelled data flow
25 to 38	Deep dive	The part the interviewer picks: dedupe, retries, provider failover, fan-out	Correct detail on one or two components
38 to 45	Wrap-up	Failure modes, monitoring (queue age, delivery rate), trade-offs, what you would build next	A list of risks and mitigations

The full answer for this exact question is in case study 17; the point here is the **shape**: requirements first, numbers second, one diagram, one deep dive, and a deliberate closing.

Phrases that signal seniority

1. "Before I design, here are the assumptions I am making; correct me if they are wrong."
2. "The bottleneck is X, so I will spend my depth there."
3. "There are two options. Option A gives up Y for Z. I would choose A because of the stated requirement."

4. "This is the failure mode, this is how we detect it, and this is the blast radius."
5. "I would start simple and name the trigger that makes us change it (a threshold in traffic, data size or team size)."

Common ways to lose the room

Mistake	Better
Naming technologies before requirements	State the need first ("ordered, durable, replayable") and then say Kafka fits
Drawing 20 boxes at once	Start with four, then add components when a requirement forces them
No numbers	One estimate of QPS and storage justifies every later choice
Treating the interviewer as an examiner	Treat them as a colleague: ask which area they want in depth
Ignoring failure	For every box, say what happens when it dies

Further Reading

- [System Design Primer](#)
- [AWS Builders' Library](#)
- [Hello Interview: system design](#)

Check Yourself

Answer in your head or on paper first, then open each answer.

- ▶ 1. List the phases of an HLD interview.
- ▶ 2. How should you use the whiteboard?
- ▶ 3. What do you do when the interviewer pushes back?
- ▶ 4. Why discuss failure modes?